



Università degli Studi di Catania
Corso di Laurea Magistrale in Informatica
Curriculum Distributed Architectures and Cybersecurity
Anno accademico 2025/2026

Ingegnerie dei Sistemi Distribuiti

Andrea Fazzina
100004892

Docente: Prof. Emiliano Alessio TRAMONTANA

Sommario

Introduzione	3
1.1 Descrizione del dataset.....	3
1.2 Identificazione dei ruoli.....	3
Il venditore (Seller)	3
L'Analista (Analyst).....	4
Architettura del Sistema.....	5
2.1 Architettura Distribuita	5
Il Server (Backend – Porta 8080).....	5
Il Client (Frontend – Porta 8081)	5
2.2 Il modulo Shared	5
2.3 Il ruolo dei DTO (Data Transfer Object) + Assembler	6
Pattern Progettuali Adottati	7
3.1 Remote Facade	7
3.2 Remote Proxy	7
3.3 Request Batching	8
Servizi Specializzati e Logica di Dominio.....	9
4.1 CatalogService.....	9
4.2 SimilarityService	9
4.3 AnalysisService	10
4.4 BatchService e Mutua Esclusione	10
Sicurezza	11
5.1 Autenticazione Stateless basata su JSON Web Token (JWT).....	11
5.2 Reference Monitor.....	13
5.3 Client Session State	12
5.4 Gestione della concorrenza.....	13

Introduzione

Lo scopo del progetto è sviluppare un'architettura software distribuita basata sul framework **Spring Boot**. L'idea concepita è quella di un servizio capace di fornire consultazione e analisi statistica su un insieme di dati automobilistici, tramite un'interfaccia web sicura e adattabile ai privilegi dell'utenza.

1.1 Descrizione del dataset

Il nucleo informativo del sistema è costituito da un dataset strutturato contenente le specifiche tecniche e commerciali di un ampio insieme di automobili. Ogni record del dataset modella un'entità automobile, tracciandone attributi fondamentali quali marca, modello, prezzo, anno di immatricolazione e vari altri parametri prestazionali.

Il sistema elabora il dataset per permettere interrogazioni mirate, filtraggi dinamici e calcoli aggregati, assicurando che le informazioni vengano fornite all'utente con il minimo overhead di rete. Il caricamento iniziale dei dati avviene in fase di bootstrap dell'applicazione server: un componente dedicato intercetta l'avvio del sistema, legge il dataset e popola il database relazionale, garantendo la consistenza iniziale e la disponibilità immediata delle informazioni per i client connessi.

1.2 Identificazione dei ruoli

L'analisi dei requisiti ha portato all'identificazione di due attori principali, caratterizzati da necessità operative distinte: il venditore e l'analista.

Il venditore (Seller)

Il venditore rappresenta l'utente operativo standard che si focalizza sulla consultazione dettagliata del catalogo auto. I suoi casi d'uso comprendono:

- Esplorazione del catalogo automobilistico completo.
- Visualizzazione della scheda di dettaglio delle singole vetture.
- Ricerca e confronto di automobili simili in base a specifiche affinità tecniche.

L'Analista (Analyst)

L'analista rappresenta l'utente interessato all'andamento di mercato, focalizzato non sulla singola entità, ma su un'aggregazione di dati. I suoi casi d'uso comprendono:

- Richiesta di analisi statistiche trasversali sull'intero dataset.
- Calcolo di metriche specifiche, come la valutazione del prezzo medio, l'impatto del deprezzamento in base al chilometraggio e altre aggregazioni matematiche.

Architettura del Sistema

Per garantire scalabilità, manutenibilità e una netta separazione delle responsabilità, il sistema si presenta come un'architettura perfettamente distribuita di tipo Client-Server. L'intero ecosistema è ingegnerizzato utilizzando Spring Boot, che fornisce le fondamenta per l'infrastruttura di rete, l'inversione del controllo e l'astrazione dell'accesso ai dati.

2.1 Architettura Distribuita

Il sistema si articola su due applicazioni Spring Boot completamente indipendenti, isolate ed eseguite su porte di rete distinte.

Il Server (Backend – Porta 8080)

È il motore applicativo del sistema che espone i servizi tramite API REST centralizzando l'intera logica di business, i calcoli per le analisi, la persistenza dei dati sul database e la validazione della sicurezza. Per la persistenza dei dati, il sistema sfrutta Spring Data JPA, che mappa le entità Java a oggetti relazionali sul database PostgreSQL. Questo approccio astrae la complessità delle query SQL, garantendo un accesso ai dati sicuro, manutenibile e fortemente tipizzato. Infine, cosa importante, il Server è totalmente agnostico rispetto alla presentazione visiva: non sa, e non deve sapere, come i dati verranno mostrati all'utente.

Il Client (Frontend – Porta 8081)

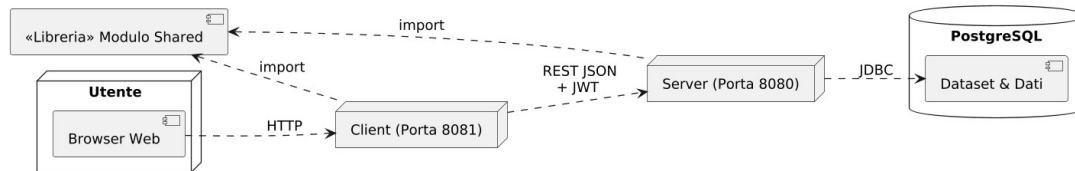
Si occupa esclusivamente dell'interfaccia utente (UI) e della gestione della sessione. Esso riceve gli input dall'utente, formula le opportune richieste HTTP verso il Server, ne attende la risposta e solo dopo si occupa del rendering delle pagine web.

Il motore responsabile di questo rendering server-side per il frontend è **Thymeleaf**, uno strumento che elabora dinamicamente i template HTML iniettando i dati estratti dal backend e permettendo di mostrare o nascondere porzioni di pagina a seconda del ruolo autorizzato.

2.2 Il modulo Shared

Avendo due applicativi separati, emerge la necessità di farli comunicare utilizzando un linguaggio comune. Di conseguenza, è stato implementato un terzo modulo indipendente denominato Shared.

Quest'ultimo agisce come una libreria comune, importata come dipendenza sia dal Server che dal Client. Il modulo contiene l'interfaccia del servizio principale e le definizioni delle strutture dati che viaggiano sulla rete. Una qualsiasi modifica all'interno del modulo Shared, viene istantaneamente percepita sia dal Client che dal Server, garantendo una totale coerenza di tutto il sistema distribuito.



2.3 Il ruolo dei DTO (Data Transfer Object) + Assembler

Un pilastro fondamentale della comunicazione tra Client e Server è l'utilizzo dei DTO, definiti all'interno del modulo Shared. Quando il Client interroga il Server, quest'ultimo non restituisce mai direttamente le Entità mappate sul database. L'impiego dei DTO apporta due vantaggi cruciali:

1. I DTO permettono di impacchettare su misura le sole informazioni strettamente necessarie al completamento di una richiesta. Ad esempio, quando il Client richiede il catalogo, il Server estrae le entità, ne mappa solo gli attributi strettamente necessari per la visualizzazione e invia il DTO. Questo previene la saturazione della banda e velocizza i tempi di risposta.
2. È buona pratica non esporre mai l'architettura interna di un database verso l'esterno, proteggendolo da eventuali accessi diretti.

Per facilitare la composizione dei DTO a partire dalla nostra Entità Automobile, è stata implementata l'adozione del pattern architetturale *Assembler* tramite la classe **AutoAssembler**, con lo scopo di convertire l'entità complessa in un DTO leggero e ottimizzato per il trasferimento di rete.

Pattern Progettuali Adottati

Il progetto adotta una serie di specifici pattern architetturali con l'obiettivo di ottimizzare la comunicazione di rete e nascondere la complessità infrastrutturale.

3.1 Remote Facade

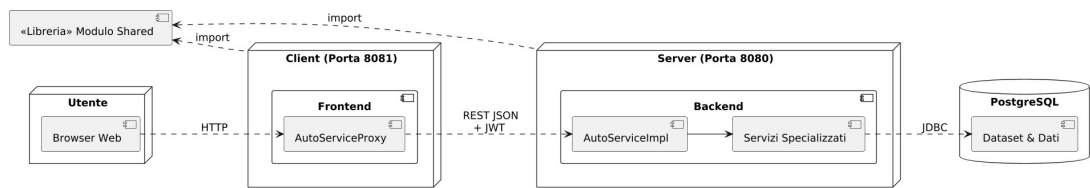
In un sistema distribuito, invocare ripetutamente metodi a grana fine (*fine-grained*) per recuperare singole informazioni genera un'eccessiva latenza dovuta al moltiplicarsi delle richieste HTTP. Per minimizzare l'overhead della comunicazione tra Client e Server, è stato implementato il pattern strutturale *Remote Facade*.

Il pattern risolve questa criticità esponendo verso l'esterno esclusivamente metodi a grana grossa (*coarse-grained*). Nel progetto, questo ruolo è ricoperto dalla classe **AutoServiceImpl**, la quale implementa l'interfaccia pubblica **AutoService** definita nel modulo Shared. Quando il Server riceve una richiesta complessa, la suddetta classe si occupa di orchestrare il recupero di tutti i dati necessari, assemblandoli internamente e restituendo al Client un unico DTO completo in una singola transazione di rete.

Oltre all'aggregazione dei dati, questa classe funge da vero e proprio punto di ingresso: si occupa infatti di gestire la logica di login e di effettuare una rigorosa verifica del ruolo dell'utente prima di consentire l'accesso alle risorse. Per preservare la leggibilità del codice ed evitare che **AutoServiceImpl** si trasformi in una classe monolitica e difficilmente manutenibile, l'elaborazione effettiva viene delegata a servizi specializzati (**CatalogService**, **SimilarityService**, **AnalysisService** e **BatchService**).

3.2 Remote Proxy

Specularmente all'adozione del Facade sul Server, lato Client la comunicazione è stata ingegnerizzata tramite il pattern *Remote Proxy*, implementato attraverso la classe **AutoServiceProxy**. L'obiettivo è fornire un surrogato locale di un oggetto risedente in uno spazio di indirizzamento differente, nascondendo la complessità dell'infrastruttura di rete. Dando l'illusione al Client di invocare un metodo in memoria locale, il proxy devia in trasparenza l'esecuzione verso il server, dove avviene la reale elaborazione logica.

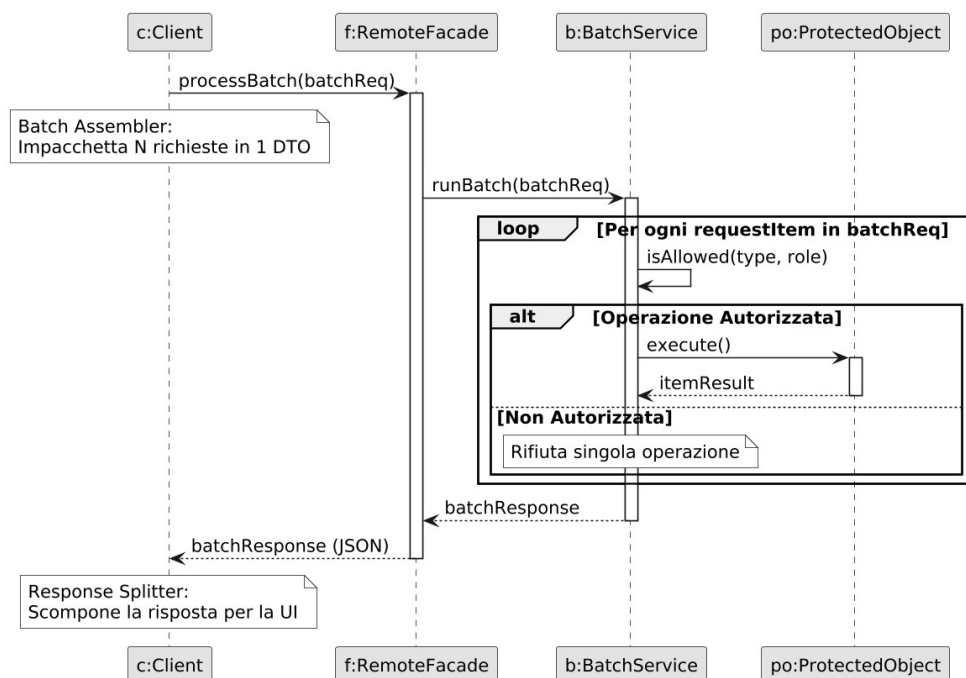


3.3 Request Batch

Per gestire scenari in cui il sistema necessita di elaborare molteplici operazioni in contemporanea, è stato adottato il pattern *Request Batch*.

In un'architettura distribuita, l'esecuzione di numerose chiamate HTTP separate verso il server rappresenta un collo di bottiglia critico. Per ovviare a questo calo prestazionale, il sistema accorpa le richieste multiple in un'unica grande transazione di rete.

Sfruttando le strutture dati definite nel modulo Shared, il Client (agendo da *Request Sender* e *Batch Assembler*) raggruppa le singole richieste all'interno di un unico payload strutturato. Il Remote Proxy si occupa quindi di inviare questo "pacchetto" consolidato al Server tramite una singola invocazione HTTP. Lato Server, il pacchetto ricevuto dalla Facade, viene disassemblato dal *Batch Processor* (**BatchService**) e le singole operazioni vengono delegate ai rispettivi servizi specializzati. Una volta terminate tutte le elaborazioni, i risultati vengono raccolti e ricompattati in un nuovo pacchetto che viene rispedito al Client con una singola operazione, dove un *Response Splitter* (**AutoServiceProxy**) li distribuirà in base alle richieste originali.



Servizi Specializzati e Logica di Dominio

Come anticipato nella descrizione del pattern Remote Facade, la classe **AutoServiceImpl** agisce da mero orchestratore. Per non appesantire il Facade, l'effettiva elaborazione dei dati e l'applicazione delle regole di business sono delegate a un livello sottostante composto da servizi altamente specializzati. Questo approccio garantisce codice modulare, testabile e facilmente manutenibile.

4.1 CatalogService

Questo servizio è il motore alla base dell'esplorazione del dataset automobilistico. Invece di estrarre l'intero database caricandolo ciecamente in memoria, il **CatalogService** sfrutta le interfacce di paginazione offerte nativamente da Spring Data JPA (tramite l'oggetto *Pageable*).

Quando il Client richiede di visualizzare il catalogo, il servizio interroga PostgreSQL richiedendo solo uno specifico sottoinsieme di record (una "pagina" alla volta). Questo approccio riduce drasticamente l'occupazione di RAM sul server, previene colli di bottiglia computazionali e minimizza i tempi di latenza sulla rete, consentendo al sistema di scalare agevolmente anche all'aumentare delle dimensioni del dataset.

4.2 SimilarityService

Dedicato principalmente a supportare i casi d'uso del Venditore (Seller), questo componente incapsula la logica per il confronto e la raccomandazione dei veicoli. Quando l'utente visualizza la scheda di dettaglio di una specifica automobile, il sistema calcola ed espone i veicoli più affini secondo diversi criteri prestabiliti (come la similarità per prezzo, per motore o per categoria di utilizzo).

Il meccanismo generico di calcolo alla base di queste raccomandazioni si fonda sull'estrazione degli attributi chiave del veicolo di riferimento. Per i valori numerici (come il prezzo o i cavalli del motore), il servizio calcola dinamicamente un intervallo di tolleranza, impostando un range percentuale di scostamento massimo rispetto al valore target. Per gli attributi testuali o categorici, si valuta invece la corrispondenza diretta. Tali vincoli matematici vengono tradotti in query personalizzate che demandano a PostgreSQL l'onere di filtrare le vetture, restituendo al frontend esclusivamente un sottoinsieme di risultati pertinenti e già scremati.

4.3 AnalysisService

Questo componente è il cuore pulsante degli strumenti a disposizione dell'Analista (Analyst). Si occupa di processare i dati grezzi per estrarre metriche di mercato ad alto valore aggiunto, come la valutazione del prezzo medio (AVG_PRICE), l'andamento del deprezzamento in base all'età (DEPRECIATION_COURSE) o l'impatto del chilometraggio sul valore del veicolo (MILEAGE_PRICE_IMPACT).

Per garantire prestazioni ottimali su grandi moli di dati, il servizio sfrutta la potenza di calcolo del motore relazionale. Le operazioni statistiche più pesanti vengono tradotte in query SQL di aggregazione (utilizzando costrutti come GROUP BY e funzioni matematiche native del database), permettendo al servizio Java di ricevere ed elaborare esclusivamente i risultati statistici finali, senza sovraccaricare l'applicativo.

4.4 BatchService e Mutua Esclusione

Il **BatchService** è il motore di esecuzione massiva che concretizza la logica dietro al pattern Request Batch. Riceve un payload compatto contenente una lista di richieste eterogenee, lo disaggrega e itera su ciascuna operazione. Per ogni singola voce del batch, il servizio invoca dinamicamente il **SimilarityService** o l'**AnalysisService** in base alla tipologia di task richiesta.

Oltre all'orchestrazione, questo servizio ricopre un ruolo fondamentale per la protezione dei dati applicando una rigorosa mutua esclusione tramite il metodo interno *isAllowed()*. Prima di eseguire qualsiasi calcolo, il servizio incrocia il ruolo dell'utente corrente (estratto in modo sicuro dal contesto del thread) con la specifica natura della richiesta. Se un utente tenta di richiedere un'operazione non consona ai propri privilegi (ad esempio un Seller che tenta di lanciare un'analisi di mercato), il servizio intercetta l'anomalia e respinge la specifica richiesta, garantendo un approccio di sicurezza granulare di tipo Zero Trust.

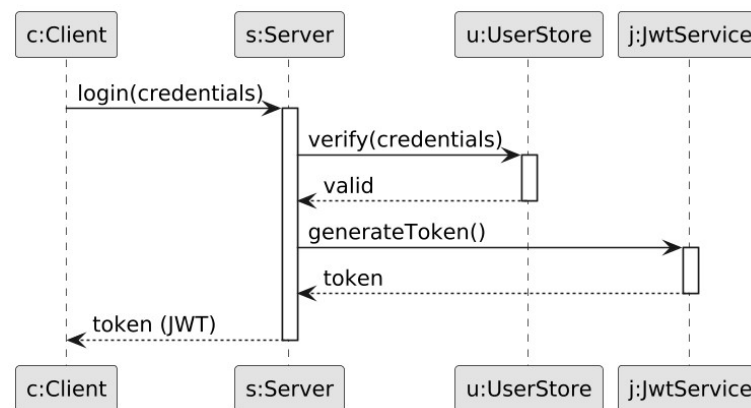
Sicurezza

5.1 Autenticazione Stateless basata su JSON Web Token (JWT)

Il sistema implementa un meccanismo di sicurezza architetturale *stateless*, eliminando la necessità di allocare e mantenere lo stato delle sessioni nella memoria del server. Questo paradigma è stato concretizzato attraverso l'adozione dello standard JSON Web Token (JWT).

Il processo di autenticazione si articola in fasi distinte: al momento del login, le credenziali immesse dall'utente vengono intercettate e verificate dal componente **UserStore**, il quale si occupa di confrontare l'hash crittografico della password ricevuta con quello memorizzato in fase di bootstrap. Per la cifratura delle password, il sistema implementa l'algoritmo memory-hard SCrypt tramite le dipendenze di Spring Security. Questa scelta architetturale garantisce una robusta protezione contro attacchi di tipo Brute Force e l'uso di Rainbow Tables, offrendo al contempo un'elevata resistenza contro i tentativi di cracking eseguiti tramite hardware dedicato (GPU).

Se l'autenticazione va a buon fine, il sistema delega al **JwtService** la generazione del token. L'algoritmo crittografico adottato per la firma dei token è HMAC-SHA256, che sfrutta una chiave crittografica simmetrica. Per prevenire vulnerabilità legate all'hardcoding, tale chiave segreta non è mai scritta direttamente nel codice sorgente, ma viene iniettata a runtime tramite l'annotazione `@Value` di Spring.



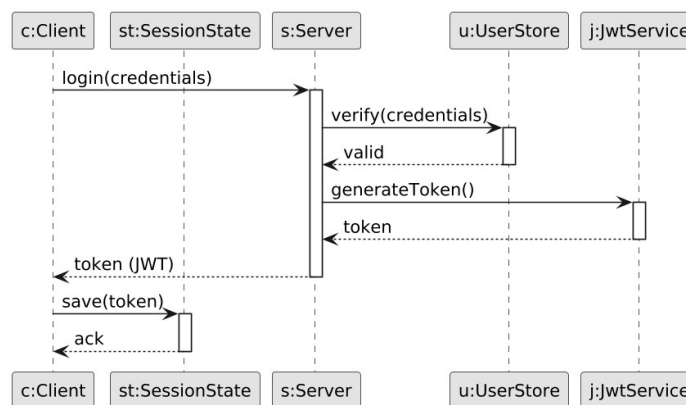
Il JWT prodotto è un oggetto auto-contenuto e crittograficamente firmato che incapsula al suo interno le informazioni essenziali dell'utente: l'*username*, e soprattutto, il *ruolo* autorizzativo (SELLER o ANALYST). Il token generato viene restituito al Client, il quale dovrà allegarlo nell'header HTTP (Authorization: Bearer <token>) di ogni successiva richiesta.

Grazie alla natura auto-contenuta del JWT, il Server è in grado di validare l'identità dell'utente e i suoi privilegi per ogni singola richiesta verificando unicamente la validità della firma digitale del token, senza dover fare affidamento ad alcuna struttura dati lato server o registri di sessione centralizzati. La validazione avviene esclusivamente in memoria, elaborando matematicamente la firma tramite la chiave segreta. A ulteriore garanzia di sicurezza, il ciclo di vita del token è strettamente regolato da una scadenza temporale (gestita tramite il parametro configurabile `jwt.expiration-ms`): una volta scaduto, il server rifiuterà la richiesta, imponendo al client di effettuare una nuova autenticazione.

5.2 Client Session State

In perfetta sinergia con l'adozione del JWT lato backend, il sistema implementa il pattern architetturale *Client Session State* per la gestione dello stato applicativo. Adottando un approccio *stateless* sul Server (porta 8080), la responsabilità di mantenere viva la sessione e ricordare l'identità dell'utente viene delegata interamente all'applicativo Client (porta 8081).

Al momento del login, il token JWT ricevuto dal Server viene memorizzato dal Client all'interno di un oggetto di stato dedicato. Come previsto dal pattern, per ogni successiva invocazione di dominio, il Client si fa carico di recuperare questo stato (il token) e di trasmetterlo al Server allegandolo alla richiesta di rete tramite l'header HTTP. Questa serializzazione del contesto utente all'interno di una stringa compatta risolve la complessità di mantenere strutture dati pesanti in memoria.



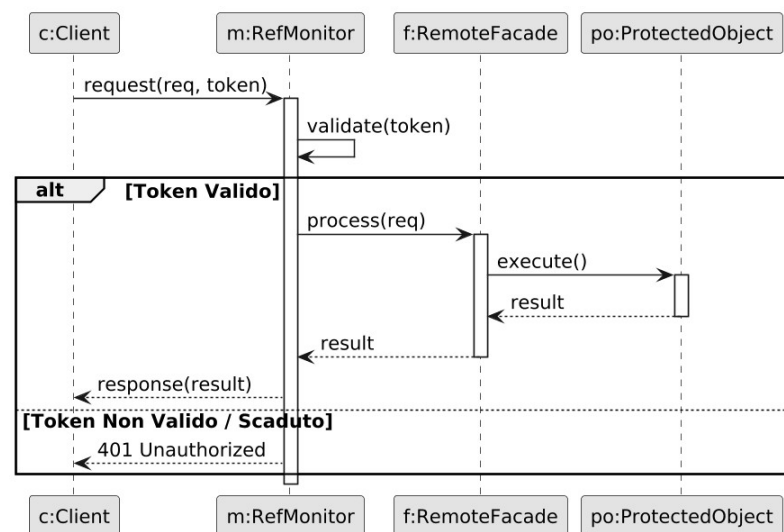
Questa scelta architetturale apporta molteplici benefici: garantisce tolleranza ai guasti (un riavvio del server non invalida le sessioni, poiché il client reimpiegherà il token valido) e previene manipolazioni esterne, in quanto la firma digitale del token impedisce all'utente di alterare fraudolentemente i propri privilegi lato client.

5.3 Reference Monitor

Per garantire che nessuna richiesta protetta possa raggiungere i servizi di dominio senza un'adeguata validazione, il sistema implementa il pattern di sicurezza *Reference Monitor*. Questo pattern impone che ogni accesso alle risorse debba passare attraverso un componente centralizzato, inaggirabile e non manomettibile.

Nel contesto di questa architettura, il ruolo di *Reference Monitor* è svolto in modo nativo dalla classe **JwtAuthFilter**. Questo filtro si posiziona all'ingresso dell'applicativo Server, agendo come un casello di controllo obbligatorio per tutte le chiamate HTTP in entrata (fatta eccezione per gli endpoint pubblici, come quello di login).

Quando una richiesta raggiunge il server, il filtro intercetta il traffico di rete ed estrapola il token JWT dall'header Authorization. Successivamente, delega la validazione crittografica e temporale al **JwtService**. Se il token risulta autentico e non scaduto, il filtro estrae le informazioni relative all'identità (lo username) e ai privilegi (il ruolo) dell'utente. In caso contrario, la richiesta viene immediatamente respinta con un codice di errore HTTP **401 Unauthorized**, bloccando l'esecuzione ancor prima che tocchi il livello applicativo.



5.4 Gestione della concorrenza

In un'architettura web gestita da Spring Boot, il server integrato (Tomcat) processa le molteplici chiamate HTTP concorrenti assegnando ciascuna richiesta a un thread separato, prelevato da un pool di risorse (*Thread Pool*). Questo approccio garantisce alte prestazioni, ma introduce una sfida architetturale: propagare l'identità dell'utente autenticato ai livelli applicativi sottostanti senza doverla passare esplicitamente come parametro, evitando così di inquinare le firme dei metodi.

La soluzione adottata nel progetto è l'implementazione della classe **CurrentUserContext**, basata sul costrutto Java *ThreadLocal*. Il *ThreadLocal* permette di definire variabili accessibili in modo globale, ma il cui valore è rigorosamente isolato e visibile esclusivamente al thread che le sta eseguendo. Nessun altro thread concorrente può leggere o sovrascrivere questi dati.

Il flusso operativo si articola come segue: non appena una richiesta HTTP supera i controlli del **JwtAuthFilter**, quest'ultimo popola il **CurrentUserContext** salvando il ruolo e lo username dell'utente direttamente nel *ThreadLocal*. Da quel momento in poi, qualsiasi servizio può interrogare il contesto per estrapolare i privilegi attuali, con la matematica certezza che i dati letti appartengano esattamente all'utente che ha originato quella specifica richiesta, scongiurando pericolose race condition.

Infine, per prevenire vulnerabilità o perdite di memoria (memory leak) causate dal riutilizzo dei thread da parte del web server, il sistema assicura che il contesto *ThreadLocal* venga rigorosamente svuotato e distrutto non appena la risposta HTTP viene restituita al Client.

